Q

Z-machine Common Save-File Format Standard

also called Quetzal:

Quetzal Unifies Efficiently The Z-Machine Archive Language

version 1.4 (13th November 1997), by Martin Frost

---

1. Conventions

1.1

A **byte** is an 8-bit unsigned quantity.

1.2

A **word** is a 16-bit unsigned quantity.

1.3

Bitfields are represented as blocks of characters, with the first character representing the most significant bit of the byte in question. Multi-bit subfields are indicated by using the same character multiple times, and values of 0 or 1 indicate that these bits are always of the specified value. Therefore a bitfield described as **010abbcc cccdd111** would be a two-byte bitfield containing four subfields, **a**, of 1 bit, **b**, 2 bits, **c**, 5 bits, and **d**, 2 bits, together with a field 'hardwired' to 010 and one to 111.

1.4

All multi-byte numbers are stored in big-endian form: most significant byte first, then in strictly descending order of significance.

1.5

The reader is assumed to already be familiar with the Z-machine; in particular its instruction set, memory map and stack conventions.

1.6

When form type names, which are four characters long, are set in running text, they're set in bold-face with spaces replaced by underscores. Thus **____** means "four spaces" and **(c)_** means three letters of the copyright notation followed by a space.

---

2. Overall structure

2.1

For the purposes of flexibility, the overall format will be a new IFF type. A standard core is defined, and customised information can be stored by specific interpreters in such a way that it can be easily read by others. The FORM type is **IFZS**.

2.2

Several chunks are defined within this document to appear in the IFZS FORM:

```
'IFhd'          5.4
'CMem'          3.7
'UMem'          3.8
'Stks'          4.10
'IntD'          7.8
```

2.3

Several chunks may also appear by convention in any IFF FORM:

```
'AUTH'          7.2, 7.3
'(c) '          7.2, 7.4
'ANNO'          7.2, 7.5
```

---

3. Content of dynamic memory

3.1

Since the contents of dynamic memory may be anything up to 65534 bytes, it is desirable to have some form of compression available as an option. Bryan Scattergood's port of ITF uses a method that is both elegant and effective, and this is the method adopted.

3.2

The data is compressed by exclusive-oring the current contents of dynamic memory with the original (from the original story file). The result is then compressed with a simple run-length scheme: a non-zero byte in the output represents the byte itself, but a zero byte is followed by a length byte, and the pair represent a block of **n+1** zero bytes, where **n** is the value of the length byte.

3.3

It is not necessary to compress optimally, if to do so would be difficult. For example, an interpreter that does not store the whole of dynamic memory in physical memory may compress a single page at a time, ignoring the possibility of a run crossing a page boundary; this case can be encoded as two adjacent runs of bytes. It is required, however, that interpreters read encoded data even if it does not happen to be compressed to their particular page-boundary preferences. This is not difficult, requiring merely the maintenance of a small amount of state (namely the current run length, if any) across page boundaries on a read.

3.4

If the decoded data is shorter than the length of dynamic memory, then the missing section is assumed to be a run of zeroes (and hence equal to the original contents of that part of dynamic memory). This permits the removal of redundant runs at the end of the encoded block; again it is not necessary to implement this on writes, but it must be understood on reads.

3.5

Two error cases are possible on reads: the decoded data may be larger than dynamic memory, and the encoded data may finish with an incomplete run (a zero byte without a length byte). These should be dealt with in whatever way seems appropriate to the interpreter writer.

3.6

Dissenting voices have suggested that compression is unnecessary in today's world of cheap storage, and so the format also includes the capability to dump the contents of dynamic memory without modification. The ability to write such files is optional; the ability to read both types is necessary. It is an error for this dump to be shorter or longer than the expected length of dynamic memory.

3.7

The IFF chunk used to contain the compressed data has type **CMem**. Its format is as follows:

3.7.1

```
4 bytes         'CMem'          chunk ID
```

3.7.2

```
4 bytes         n               chunk length
```

3.7.3

```
n bytes         ...             compressed data as above
```

3.8

The chunk used to contain the uncompressed data has type **UMem**. It has the format:

3.8.1

```
4 bytes         'UMem'          chunk ID
```

3.8.2

```
4 bytes         n               chunk length
```

3.8.3

```
n bytes         ...             simple dump of dynamic memory
```

---

4. Content of stacks

4.1

One of the biggest differences between current interpreters is how they handle the Z-machine's stacks. Conceptually, there are two, but many interpreters store both in the same array. This format stores both in the same IFF chunk, which has chunk ID **Stks**.

4.2

The IFF format includes a length field on each chunk, so we can write only the used portion of the stacks, to save space. The least recent frames on the stacks are saved first, to ensure that the missing part appears at the end of the data in the file.

4.3

Each frame has the format:

4.3.1

```
3 bytes        ...            return PC (byte address)
```

4.3.2

```
1 byte         000pvvvv       flags
```

4.3.3

```
1 byte         ...            variable number to store result
```

4.3.4

```
1 byte         0gfedcba       arguments supplied
```

4.3.5

```
1 word         n              number of words of evaluation
                              stack used by this call
```

4.3.6

```
v words        ...            local variables
```

4.3.7

```
n words        ...            evaluation stack for this call
```

4.4

The return PC is a byte offset from the start of the story file.

4.6

The p flag is set on calls made by **call_xN** (discard result), in which case the variable number is meaningless (and should be written as a zero).

4.7

Assigning each of the possible 7 supplied arguments a letter a-g in order, each bit is set if its respective argument is supplied. The evaluation stack count allows the reconstruction of the chain of frame pointers for all possible stack models. Words on the evaluation stack are also stored least recent first.

4.8

Although some interpreters may impose an arbitrary limit on the size of the stacks (such as ZIP's 1024-word total stack size), others may not, or may set larger limits. This means that the size of a stack dump may be larger than will fit. If you cannot dynamically resize your stack you must trap this as an error.

4.9

The stack pointer itself is not stored anywhere in the save file, except implicitly, as the top frame on the stack will be the last saved.

4.10

The chunk itself is simply a sequence of frames as above:

4.10.1

```
    4 bytes        'Stks'         chunk ID
```

4.10.2

```
    4 bytes        n              chunk length
```

4.10.3

```
    n bytes        ...            frames (oldest first)
```

4.11

In Z-machine versions other than V6 execution starts at an address rather than at a routine, and therefore data can be pushed on the evaluation stack without anything being on the call stack. Therefore, in all versions other than V6 a dummy stack frame must be stored as the first in the file (the oldest chunk).

4.11.1

The dummy frame has all fields set to zero except **n**, the amount of evaluation stack used. Note that this may also be zero if the game does not use any evaluation stack at the top level.

4.11.2

This frame must be written even if no evaluation stack is used at the top level, and therefore interpreters may assume its presence on savefiles for V1-5 and V7-8 games.

---

5. Associated story file

5.1

We now come to one of the most difficult (yet most important) parts of the format: how to find the story file associated with this save file, or the related (but easier) problem of checking whether a given save file belongs to a given story.

5.2

Considering the easier second problem first, the actual name of the story file is often not much use. Firstly, filenames are highly dependent on the operating system in use, and secondly, many original Infocom story files were called simply 'story.data' or similar.

5.3

The method most existing interpreters use is to compare the variables at offsets $2, $12, and $1C in the header (that is, the release number, the serial number and the checksum), and refuse to load if they differ. These variables are duplicated in the file (since the header will be compressed with the rest of dynamic memory).

5.4

This data will be stored in a chunk of type **IFhd**. This chunk must come before the **[CU]Mem** and **Stks** chunks to save interpreters the trouble of decoding these only to find that the wrong story file is loaded. The format is:

5.4.1

```
4 bytes         'IFhd'          chunk ID
```

5.4.2

```
4 bytes         13              chunk length
```

5.4.3

```
1 word          ...             release number ($2 in header)
```

5.4.4

```
6 bytes         ...             serial number ($12 in header)
```

5.4.5

```
1 word          ...             checksum ($1C in header)
```

5.4.6

```
3 bytes         ...             PC (see 5.8)
```

5.5

If the save file belongs to an old game that does not have a checksum, it should be calculated in the normal way from the original story file when saving. It is possible that a future version of this format may have a larger **IFhd** chunk, but the first 13 bytes will always contain this data, and if the other chunks described herein are present they will be guaranteed to contain the data specified.

5.6

The first problem (of trying to find a story file given only a save file) cannot really be solved in an operating-system independent manner, and so there is provision for OS-dependent chunks to handle this.

5.7

It should be noted that the current state of the **IFhd** chunk means it has odd length (13 bytes). It should, of course, be written with a pad byte (as mentioned in 8.4.1).

5.8

The value of the PC saved in the chunk depends on the version of the Z-machine which the story runs on.

5.8.1

On Z-machine versions 3 and below, the **save** instruction takes a branch depending on the success of the save. The saved PC points to the one or two bytes which describe this branch.

5.8.2

On versions 4 and above, the **save** instruction stores a value depending on the success of the save. The saved PC points to the single byte describing where to store the result.

5.8.3

This behaviour differs from that specified by previous versions of this standard, but the behaviour expected there would be difficult to implement in existing interpreters. The situation has been complicated as the patches available for the Zip interpreter did not correctly implement the previous standard; instead, they behaved as specified here.

---

6. Miscellaneous

6.1

It must be specified exactly what the magic cookie returned by **catch** is, since this value can be stored in any random variable, on the evaluation stack, or indeed anywhere in memory.

6.2

For greatest independence of internal interpreter implementation, **catch** is hereby specified to return the number of frames currently on the system stack. This makes **throw** slightly inefficient on many interpreters (a current frame count can be maintained internally to avoid problems with **catch**), but this is unavoidable without using two stacks and a fixed-size activation record (always 15 local variables). Since most applications of **catch**/**throw** do not unwind enormous depths, (and they are somewhat infrequent), this should not be too much of a problem.

6.3

The numbers of pictures and sounds do not need specification, since they are requested by number by the story file itself.

---

7. Extensions to the format

7.1

One of the advantages of the IFF standard is that extra chunks can be added to the format to extend it in various ways. For example, there are three standard chunk types defined, namely **AUTH**, **(c)_**, and **ANNO**.

7.2

AUTH, **(c)_**, and **ANNO** chunks all contain simple ASCII text (all characters in the range 0x20 to 0x7E).

7.2.1

The only indication of the length of this text is the chunk length (there is no zero byte termination as in C, for example).

7.2.2

The IFF standard suggests a maximum of 256 characters in this text as it may be displayed to the user upon reading, although it could get longer if required.

7.3

The **AUTH** chunk, if present, contains the name of the author or creator of the file. This could be a login name on multi-user systems, for example. There should only be one such chunk per file.

7.4

The **(c)_** chunk contains the copyright message (date and holder, without the actual copyright symbol). This is unlikely to be useful on save files. There should only be one such chunk per file.

7.5

The **ANNO** chunk contains any textual annotation that the user or writing program sees fit to include. For save files, interpreters could prompt the user for an annotation when saving, and could write an **ANNO** with the score and time for V3 games, or a chunk containing the name/version of the interpreter saving it, and many other things.

7.6

The **ANNO**, **(c)_** and **AUTH** chunks are all user-level information. Interpreters must not rely on the presence or absence of these chunks, and should not store any internal magic that would not make sense to a user in them.

7.7

These chunks should be either ignored or (optionally) displayed to the user. **(c)_** chunks should be prefixed with a copyright symbol if displayed.

7.8

The save-file may contain interpreter-dependent information. This is stored in an **IntD** chunk, which has format:

7.8.1

```
4 bytes         'IntD'          chunk ID
```

7.8.2

```
4 bytes         n               chunk length
```

7.8.3

```
4 bytes         ...             operating system ID
```

7.8.4

```
1 byte          000000sc        flags
```

7.8.5

```
1 byte          ...             contents ID
```

7.8.6

```
2 bytes         0               reserved
```

7.8.7

```
4 bytes         ...             interpreter ID
```

7.8.8

```
n-12 bytes      ...             data
```

7.9

The operating system and interpreter IDs are normal IFF 4-character IDs in form. Please register IDs used with Martin Frost (at the email address given below) so that this can be managed sensibly. They can then be added to future versions of this specification, and contents IDs can be assigned.

7.10

If the s flag is set, then the contents are only meaningful on the same machine/network on which they were saved. This covers filenames and similar things. How to handle checking if this is indeed the same machine is an open question, and beyond the scope of this document. It is certainly true, however, that if the operating system ID does not match the current system and this bit is set, then the chunk should not be copied.

7.11

If the c flag is set, the contents should not be copied when loading and saving a game--they are only relevant to the exact current state of play as stored in the file. The data need not be copied even if this flag is clear, but must not be copied if it is set.

7.12

If the interpreter ID is ____ (four spaces), then the chunk contains information useful to *all* interpreters running on a particular system. This can store a magical OS-dependent reference to the original story file, which need not worry about vagaries of filename handling on more than one system. This chunk may contain anything that can be put in a file and retrieved intact. If the file is restored on a suitable system this can be used to do Good Things.

7.13

If the operating-system ID is ____, then the chunk contains data useful to *all* ports of a particular interpreter. This may or may not be useful.

7.14

The interpreter and operating-system IDs may not both be ____. This should not be neccessary.

7.15

If neither ID is ____, the contents are meaningful only to a particular port of a particular interpreter. Save-file specific preferences probably fall into this category.

7.16

The contents ID will be defined when chunk IDs are picked. Its purpose is to allow multiple chunks to be written containing different data, which is necessary if they need different settings of the c and s flags.

7.17

These extensions add no overhead to interpreters which choose not to handle them, except for larger save files and more chunks to skip when reading files written on another program. Interpreters are not expected to preserve these optional chunks when files are re-saved, although some may be copied, at the option of the interpreter writer or user.

7.18

The only required chunks are **IFhd**, either **CMem** or **UMem**, and **Stks**. The total overhead to a save file is 12 bytes plus 8 for each chunk; in the minimal case (**IFhd**, **[CU]Mem**, **Stks** = 3 chunks), this comes to 36 bytes.

7.19

The following operating system IDs have been registered:

7.19.1

```
'DOS '          MS-DOS (also PC-DOS, DR-DOS)
```

7.19.2

```
'MACS'          Macintosh
```

7.19.3

```
'UNIX'          Generic UNIX
```

7.20

The following interpreter IDs have been registered:

7.20.1

```
'JZIP'          JZIP, the enhanced ZIP by John Holder
```

7.21

The following extension chunks have been registered to date:

```
System ID       Interp ID       Content ID      Section
'MACS'          '    '          0               7.22
```

7.22

The following chunk has been registered for MacOS, to enable a Macintosh interpreter to find a story file given a save file using the System 7 ResolveAlias call. The MacOS alias record can be of variable size: the actual size can be calculated from the chunk size. Aliases are valid only on the same network as they were saved.

7.22.1

```
4 bytes       'IntD'        chunk ID
```

7.22.2

```
4 bytes       n             chunk length (variable)
```

7.22.3

```
4 bytes       'MACS'        operating system ID: MacOS
```

7.22.4

```
1 byte        00000010      flags (s set; c clear)
```

7.22.5

```
1 byte        0             contents ID
```

7.22.6

```
2 bytes       0             reserved
```

7.22.7

```
4 bytes       '    '        interpreter ID: any
```

7.22.8

```
n-12 bytes    ...           MacOS alias record referencing
                            the story file; from NewAlias
```

7.19.9

Alias records are of variable length, reflected in the chunk length; they are only valid on the same network they were created.

---

8. Introduction to the IFF format

8.1

This is based on the official IFF standards document, which is rather long and contains much that is irrelevant to the task in hand. Feel free to mail me (i.e. Martin Frost) if there are errors, inconsistencies, or omissions. For the inquisitive, a document containing much of the original standard, including the philosophy behind the structure, can be found at http://www.concentric.net/~Bradds/iff.html

8.2

IFF stands for "Interchange File Format", and was developed by a committee consisting of people from Commodore-Amiga, Electronic Arts and Apple. It draws strongly on the Macintosh's concept of resources.

8.3

The most fundamental concept in an IFF file is that of a chunk.

8.3.1

A chunk starts with an ID and a length.

8.3.2

The ID is the concatenation of four ASCII characters in the range 0x20 to 0x7E.

8.3.3

If spaces are present, they must be the last characters (there must be no printing characters after a space).

8.3.4

IDs are compared using a simple 32-bit equality test - note that this implies case sensitivity.

8.3.5

The length is a 32-bit unsigned integer, stored in big-endian format (most significant byte, then second most, and so on).

8.4

After the ID and length, there follow (length) bytes of data.

8.4.1

If length is odd, these are followed by a single zero byte. This byte is *not* included in the chunk length, but it is very important, as otherwise many 68000-based readers will crash.

8.5

A simple IFF file (such as the ones we will be considering) consists of a *single* chunk of type **FORM**.

8.5.1

The contents of a **FORM** chunk start with another 4-character ID.

8.5.2

This ID is also the concatenation of four characters, but these characters may only be uppercase letters and trailing spaces. This is to allow the **FORM** sub-ID to be used as a filename extension.

8.6

After the sub-ID comes a concatenation of chunks. The interpretation of these chunks depends on the **FORM** sub-ID (in this proposal, the sub-ID is **IFZS**), except that a few chunk types always have the same meaning (notably the **AUTH**, **(c)_** and **ANNO** chunks described in section 7). For reference, the other reserved types are: **FOR[M1-9]**, **CAT[1-9]**, **LIS[T1-9]**, **TEXT**, and **____** (that is, four spaces).

8.7

Each of these chunks may contain as much data as required, in whatever format is required.

8.8

Multiple chunks with the same ID may appear; the interpretation of such chunks depends on the chunk. For example, multiple **ANNO** chunks are acceptable, and simply refer to multiple annotations. If more than one chunk of a certain type is found, when the reader was only expecting one, (for example, two **IFhd** chunks), the later chunks should simply be ignored (hopefully with a warning to the user).

8.9

Indeed, skipping is the expected procedure for dealing with any unknown or unexpected chunk.

8.10

Certain chunks may be compulsory if the **FORM** is meaningless without them. In this case the **IFhd**, **[CU]Mem** and **Stks** are compulsory.

---

9. Resources available

9.1

A set of patches exists for the Zip interpreter, adding Quetzal support. They can be obtained from:

http://www.geocities.com/SiliconValley/Vista/6631/

9.2

A utility, **ckifzs** is available as C source code to check the validity of generated save files. A small set of correct Quetzal files are also available. These may be of use in debugging an interpreter supporting Quetzal. These may be obtained from the web page mentioned in 9.1.

9.3

This document is updated whenever errors are noticed or new extension chunks are registered. The latest version will always be available from the above web page. The latest revision designated stable (currently version 1.3) will be in the IF archive, ifarchive.org, in the directory if-archive/infocom/interpreters/specification/.

9.4

This document is itself available in a number of forms. The base version is in preformatted ASCII text, but there is also a PDF version (converted by John Holder) and this HTML version (converted by Graham Nelson). Links to all of these may be found on the web page.

9.5

A few interpreters support Quetzal; details will appear here as they become available.

---

10. Credits

10.1

This standard was created by Martin Frost (email: mdf@doc.ic.ac.uk). Comments and suggestions are always welcome (and any errors in this document are entirely my own, or those of the HTML typesetter, Graham Nelson).

10.2

The following people have contributed with ideas and criticism (in alphabetical order): King Dale, Marnix Klooster, Graham Nelson, Andrew Plotkin, Matthew T. Russotto, Bryan Scattergood, Miron Schmidt, Colin Turnbull, John Wood.

Remarks

Queztal is not a compulsory part of the Z-Machine Standard, since it does not have implications for the behaviour of story files, but it is attached to the HTML copy of the Standard as a highly recommended "optional extra".

Links to related sections of the Z-Machine Standards Document:

Contents / Section 6.1 on the saved state

Opcodes: **save** / **save_undo** / **restore** / **restore_undo** / **catch** / **throw**
